

LAPORAN PRAKTIKUM

RESTFUL API MURNI DENGAN EXPRESS.JS

Pengembangan Aplikasi Web II



Disusun oleh:

Nama : Sunita Aqila

NIM : 2428240072

Kelas : 5I5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Sunita Aqila
NPM	2428240072
Kelas	SI5B
Pertemuan Ke-	4
Topik / Judul Praktikum	RESTful API Murni dengan Express.js
Nomor Topik	16
Nama Topik	Kampus – Ruang Kelas
Resource	/classrooms
Tanggal Pelaksanaan	06 Oktober 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).

1. Membangun project RESTful API menggunakan Express.js dengan struktur project yang sesuai, dapat dijalankan pada lingkungan lokal, serta dapat di-deploy pada Vercel.
2. Menerapkan routing RESTful menggunakan endpoint yang sesuai dengan konvensi REST, termasuk penggunaan route parameter :id, query parameter untuk filter data, serta penanganan endpoint yang tidak ditemukan.
3. Menerapkan HTTP method sesuai fungsinya, yaitu GET untuk mengambil data, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data, disertai validasi field wajib serta penanganan kondisi data yang tidak ditemukan.
4. Menerapkan response API dalam format JSON murni dengan penggunaan HTTP status code yang sesuai, yaitu 200, 201, 400, dan 404, serta struktur response {status, message, data} pada operasi yang memerlukannya.

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 1 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

1. RESTful API dan Resource

RESTful API merupakan pendekatan pembuatan API yang menggunakan prinsip REST untuk memungkinkan pertukaran data antara client dan server. Data direpresentasikan sebagai resource yang diakses melalui URL tertentu. Pada praktikum ini, resource yang digunakan adalah ruang kelas dengan endpoint `/classrooms` (Smalley, 2025)

2. CRUD

CRUD merupakan operasi dasar pengelolaan data yang terdiri dari Create, Read, Update, dan Delete. Pada RESTful API, operasi tersebut diterapkan menggunakan POST untuk membuat data, GET untuk membaca data, PUT untuk memperbarui data, dan DELETE untuk menghapus data (Mozilla, HTTP request methods, n.d.)

3. HTTP Request, Response, Method dan Status Code

HTTP request merupakan permintaan dari client kepada server, sedangkan response merupakan hasil yang dikembalikan server. HTTP method menentukan tindakan terhadap resource, seperti GET, POST, PUT, dan DELETE. Status code menunjukkan hasil request, yaitu 200 untuk berhasil, 201 untuk data berhasil dibuat, 400 untuk request tidak valid, dan 404 untuk data atau endpoint yang tidak ditemukan. (Mozilla, HTTP messages, n.d.)

4. JSON

JSON (JavaScript Object Notation) merupakan format pertukaran data yang memiliki struktur sederhana dan dapat digunakan untuk merepresentasikan object, array, string, angka, boolean, dan null. Pada praktikum ini, JSON digunakan pada request POST/PUT dan seluruh response API (Mozilla, JSON, n.d.)

5. Routing, Route Parameter dan Query Parameter

Routing menentukan bagaimana server menangani request berdasarkan HTTP method dan URL. Express mendukung route parameter seperti `:id` melalui `req.params` dan query parameter melalui `req.query`. Pada praktikum ini, `/classrooms/:id` digunakan untuk mengakses data berdasarkan ID, sedangkan `/classrooms?gedung=A` digunakan untuk melakukan filter berdasarkan Gedung (Express.js, Routing, n.d.)

6. Middleware dan Validasi

Middleware merupakan fungsi yang dijalankan dalam proses request dan response. Pada praktikum ini, `express.json()` digunakan untuk membaca data JSON dari request body, sedangkan validasi digunakan untuk memastikan field wajib kodeRuang, gedung, dan kapasitas telah diisi. Jika data wajib tidak lengkap, server memberikan status 400. (Express.js, n.d.)

7. In-Memory Data

In-memory data merupakan data yang disimpan sementara di dalam memori aplikasi tanpa menggunakan database. Sesuai ketentuan praktikum, data ruang kelas disimpan dalam array `classrooms` di `app.js` dengan minimal tiga data awal. Data tersebut hanya tersedia selama aplikasi berjalan. (Express.js, Using middleware, n.d.)

8. Express.js

Express.js merupakan framework Node.js yang digunakan untuk membangun aplikasi web dan API. Express menyediakan fitur routing dan middleware untuk menangani request serta response dari client

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Digunakan untuk menjalankan aplikasi berbasis JavaScript pada sisi server
2	Visual Studio Code	Digunakan sebagai code editor untuk mengembangkan project
3	Express.js	Framework Node.js untuk membangun RESTful API
4	npm	Digunakan untuk mengelola package dan dependency project
5	Nodemon	Digunakan untuk membantu menjalankan ulang server selama proses pengembangan
6	Thunder Client	Digunakan untuk mengirim request serta menguji response dan HTTP status code setiap endpoint.

7	Vercel	Digunakan untuk melakukan deployment RESTful API agar dapat diakses melalui URL publik.
8	Browser	Digunakan untuk mengakses dan menguji endpoint API
9	GitHub	Digunakan sebagai repository untuk menyimpan dan mengelola project secara online.

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. Inisialisasi Project Express.js

Praktikum diawali dengan membuat project Node.js baru sebagai tempat pengembangan RESTful API. Project dibuat menggunakan Node.js dan dikelola menggunakan npm. Setelah folder project dibuat, dilakukan inisialisasi project dengan perintah `npm init -y` sehingga terbentuk file `package.json`.

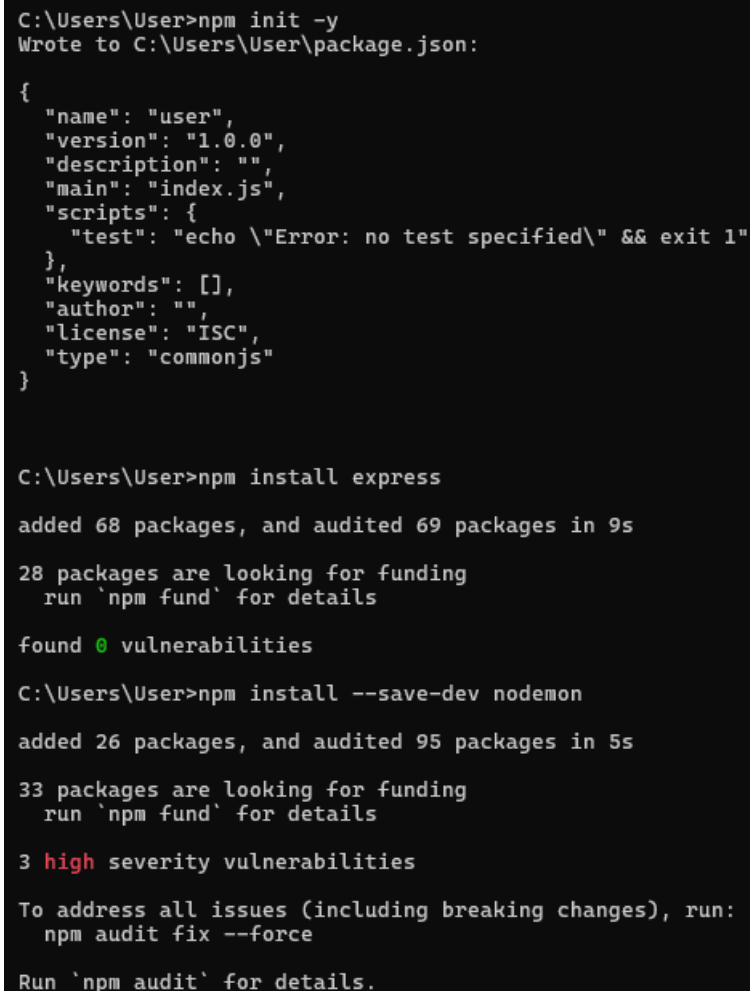
Selanjutnya, Express.js diinstal sebagai dependency utama untuk membangun RESTful API. Nodemon juga diinstal sebagai dev dependency untuk membantu menjalankan ulang server secara otomatis ketika terdapat perubahan pada kode.

Perintah yang digunakan:

```
npm init -y
```

```
npm install express
```

```
npm install --save-dev nodemon
```



```
C:\Users\User>npm init -y
Wrote to C:\Users\User\package.json:

{
  "name": "user",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

C:\Users\User>npm install express

added 68 packages, and audited 69 packages in 9s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\User>npm install --save-dev nodemon

added 26 packages, and audited 95 packages in 5s

33 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

Gambar 1. Proses inisialisasi project dan instansi Express.js serta Nodemon

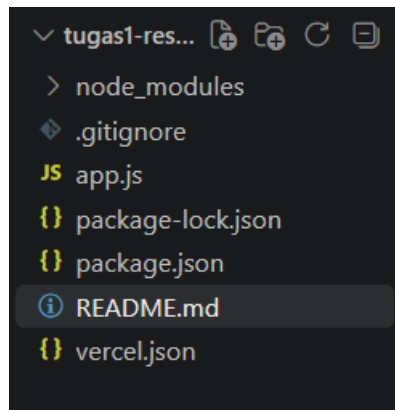
2. Membuat Struktur Project

Setelah proses instalasi selesai, project disusun dengan struktur sederhana sesuai ketentuan praktikum. Seluruh kode RESTful API ditempatkan dalam satu file utama, yaitu app.js. Struktur project yang digunakan adalah:

tugas1-restful-2428240072/

```
|— node_modules/  
|— app.js  
|— package.json  
|— package-lock.json  
|— vercel.json  
|— .gitignore
```

Folder node_modules tidak diunggah ke repository GitHub karena sudah dikecualikan melalui file .gitignore.



Gambar 2. Struktur folder project pada Visual Studio Code

3. Membuat Konfigurasi Express.js

Pada tahap ini, Express.js diimpor ke dalam app.js, kemudian dibuat instance aplikasi Express. Middleware `express.json()` digunakan agar server dapat membaca data JSON yang dikirim melalui request body.

Kode yang digunakan:

```
const express = require('express');  
const app = express();  
app.use(express.json());
```

Middleware tersebut diperlukan terutama pada request POST dan PUT karena data ruang kelas dikirim dalam format JSON.

4. Membuat Endpoint Dasar

Karena praktikum menggunakan penyimpanan in-memory dan tidak menggunakan database, data ruang kelas disimpan dalam sebuah array bernama `classrooms`.

Data awal dibuat sebanyak tiga record sesuai ketentuan minimal praktikum. Setiap data memiliki atribut `id`, `kodeRuang`, `gedung`, `lantai`, `kapasitas`, dan ada `Proyektor`.

Kode yang digunakan:

```
1  const express = require('express');
2  const app = express();
3  app.use(express.json());
4
5  // Data ruang kelas
6  let classrooms = [
7    {
8      id: 1,
9      kodeRuang: "A-301",
10     gedung: "A",
11     lantai: 3,
12     kapasitas: 40,
13     adaProyektor: true
14   },
15   {
16     id: 2,
17     kodeRuang: "A-406",
18     gedung: "A",
19     lantai: 4,
20     kapasitas: 40,
21     adaProyektor: true
22   },
23   {
24     id: 3,
25     kodeRuang: "B-609",
26     gedung: "B",
27     lantai: 6,
28     kapasitas: 80,
29     adaProyektor: false
30   }
31 ];
32
33 let nextId = 4;
```

Gambar 3. Data awal ruang kelas pada app.js

Variabel nextId digunakan untuk memberikan ID baru ketika data ruang kelas ditambahkan melalui endpoint POST.

5. Membuat Endpoint GET /

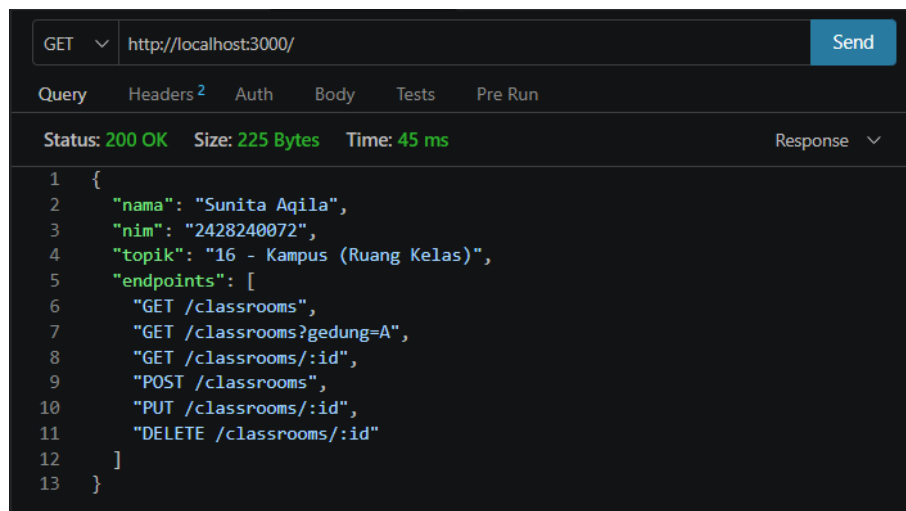
Endpoint GET / digunakan sebagai halaman awal API. Endpoint ini memberikan informasi mengenai identitas pembuat, NIM, topik yang digunakan, serta daftar endpoint yang tersedia.

Kode yang digunakan:

```
// GET /
app.get('/', (req, res) => {
  res.status(200).json({
    nama: "Sunita Aqila",
    nim: "2428240072",
    topik: "16 - Kampus (Ruang Kelas)",
    endpoints: [
      "GET /classrooms",
      "GET /classrooms?gedung=A",
      "GET /classrooms/:id",
      "POST /classrooms",
      "PUT /classrooms/:id",
      "DELETE /classrooms/:id"
    ]
  });
});
```

Gambar 4. Kode yang digunakan untuk endpoint GET /

Response diberikan dalam format JSON dengan status HTTP 200.



Gambar 5. Pengujian GET/ pada Thunder Client

6. Membuat Endpoint GET /Classrooms

```
52 // GET /classrooms
53 // GET /classrooms?gedung=A
54 app.get('/classrooms', (req, res) => {
55   const gedung = req.query.gedung;
56   if (gedung) {
57     const hasil = classrooms.filter(
58       (classroom) => classroom.gedung === gedung
59     );
60     return res.status(200).json(hasil);
61   }
62
63   res.status(200).json(classrooms);
64 });
```

Gambar 6. Pengujian GET/Classrooms pada Thunder Client

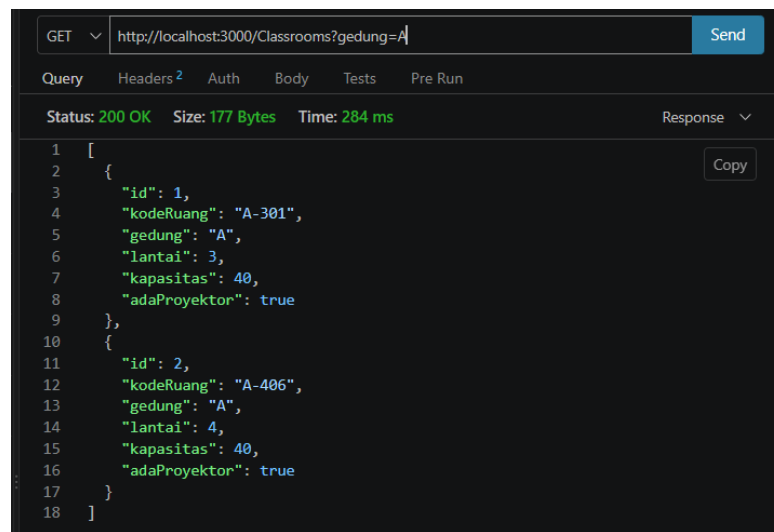
Jika request tidak memiliki query parameter gedung, server mengembalikan seluruh data dalam array. Response berhasil menggunakan status 200.

7. Membuat Filter GET/Classrooms?gedung=A

Selain mengambil seluruh data, endpoint /classrooms digunakan untuk melakukan filter berdasarkan gedung. Filter dilakukan menggunakan query parameter gedung.

Contoh request: GET /classrooms?gedung=A

Nilai A dibaca menggunakan req.query.gedung, kemudian data difilter menggunakan fungsi filter(). Dengan demikian, hanya ruang kelas yang memiliki nilai gedung sama dengan A yang dikembalikan.



Gambar 7. Pengujian GET/Classrooms?gedung=A pada Thunder Client

8. Membuat Endpoint GET/Classrooms/:id

Endpoint ini digunakan untuk mengambil satu data ruang kelas berdasarkan ID. ID diperoleh melalui route parameter :id dan dibaca menggunakan req.params.id.

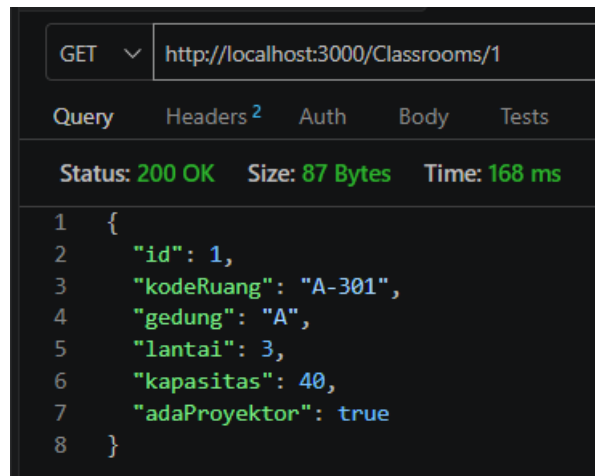
Kode yang digunakan:

```

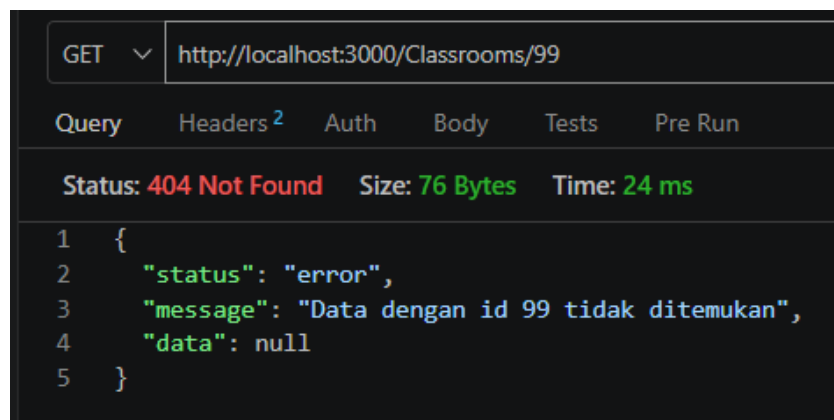
// GET /classrooms/:id
app.get('/classrooms/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const classroom = classrooms.find(
    (classroom) => classroom.id === id
  );
  if (!classroom) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null
    });
  }
  res.status(200).json(classroom);
});
  
```

Gambar 8. Kode yang digunakan pada GET/Classrooms?gedung=A

Jika data ditemukan, server memberikan response berupa object ruang kelas dengan status 200. Jika ID tidak ditemukan, server memberikan status 404.



Gambar 9. Kode yang digunakan pada GET/Classrooms/1



Gambar 10. Kode yang digunakan pada GET/Classrooms/99

9. Membuat Endpoint POST /classrooms

Endpoint POST /classrooms digunakan untuk menambahkan data ruang kelas baru. Data dikirim melalui request body dalam format JSON. Field yang digunakan adalah:

- kodeRuang
- gedung
- lantai
- kapasitas
- adaProyektor

Contoh request body:

```
{
  "kodeRuang": "C-101",
  "gedung": "C",
  "lantai": 1,
  "kapasitas": 50,
  "adaProyektor": true }
```

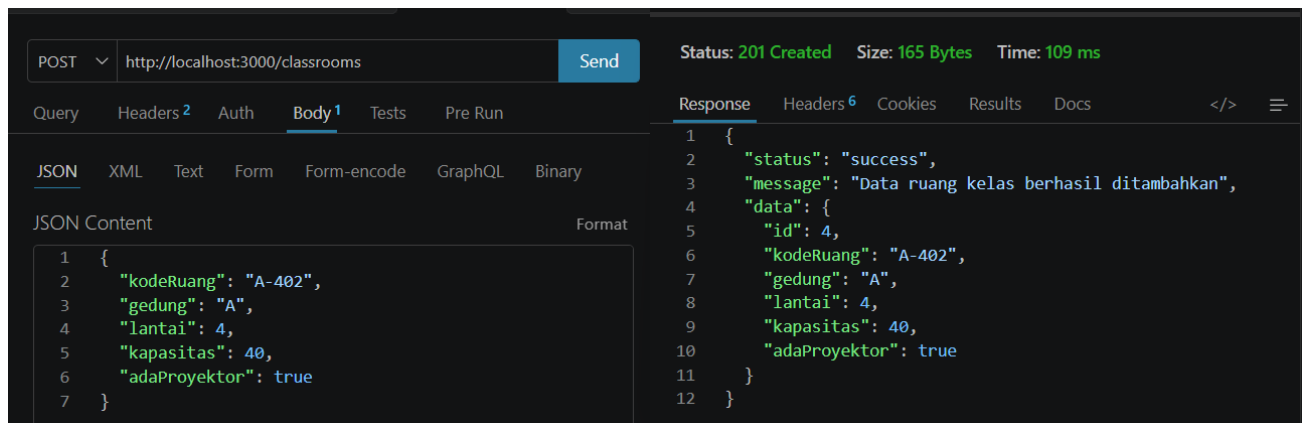
```

app.post('/classrooms', (req, res) => {
  const {
    kodeRuang,
    gedung,
    lantai,
    kapasitas,
    adaProyektor
  } = req.body;
  if (!kodeRuang || !gedung || kapasitas === undefined) {
    return res.status(400).json({
      status: "error",
      message: "kodeRuang, gedung, dan kapasitas wajib diisi",
      data: null
    });
  }
  const classroomBaru = {
    id: nextId++,
    kodeRuang: kodeRuang,
    gedung: gedung,
    lantai: lantai,
    kapasitas: kapasitas,
    adaProyektor: adaProyektor
  };
  classrooms.push(classroomBaru);
  res.status(201).json({
    status: "success",
    message: "Data ruang kelas berhasil ditambahkan",
    data: classroomBaru
  });
});

```

Gambar 11. Kode yang digunakan pada GET/Classrooms/99

Jika data berhasil ditambahkan, server memberikan status 201 karena terdapat resource baru yang berhasil dibuat.



Gambar 12. Pengujian POST /classrooms dengan data lengkap

10. Melakukan Validasi Data

Validasi dilakukan untuk memastikan field wajib telah diisi sebelum data diproses. Field wajib pada resource ruang kelas adalah kodeRuang, gedung, dan kapasitas. Jika salah satu field wajib tidak diberikan, server mengembalikan status 400 dengan response JSON.

Contoh response:

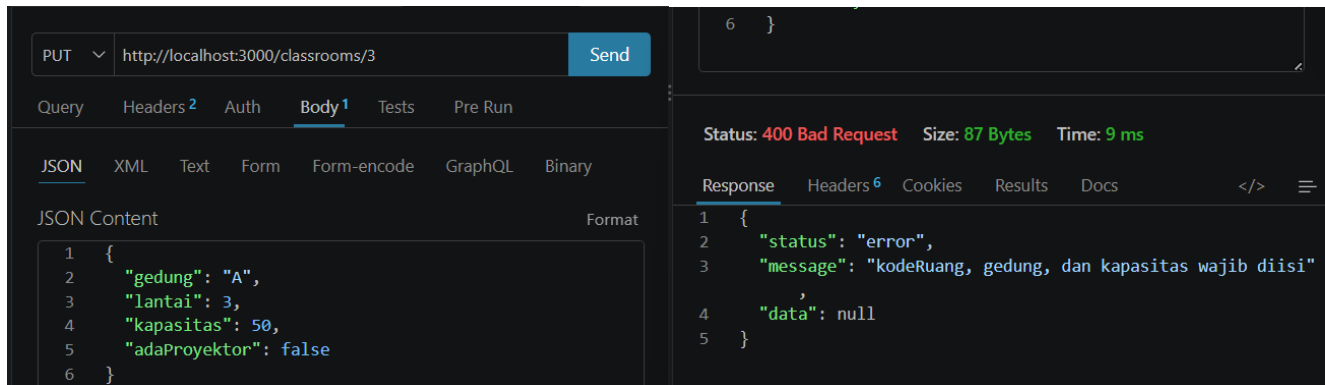
```

{
  "status": "error",
  "message": "kodeRuang, gedung, dan kapasitas wajib diisi",

```

```
"data": null
}
```

Validasi yang sama juga diterapkan pada endpoint PUT agar data yang diperbarui tetap memenuhi ketentuan field wajib.



Gambar 13. Pengujian POST /classrooms dengan data tidak lengkap

11. Membuat Endpoint PUT /classrooms/:id

Endpoint PUT digunakan untuk memperbarui data ruang kelas berdasarkan ID. Kode yang digunakan:

```
app.put('/classrooms/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = classrooms.findIndex(
    (classroom) => classroom.id === id
  );

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: "Data dengan id ${id} tidak ditemukan",
      data: null
    });
  }

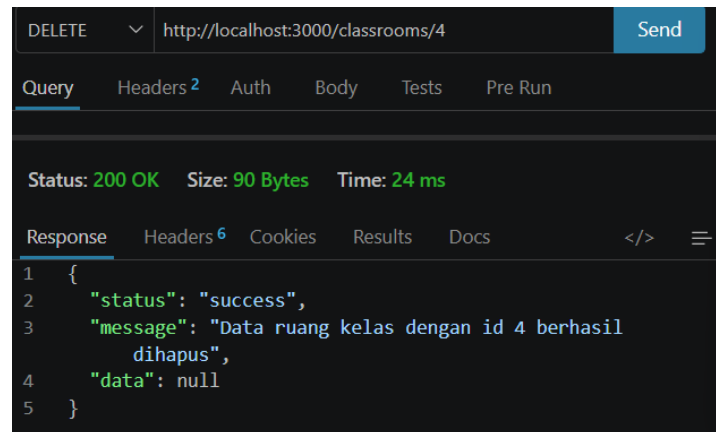
  const {
    kodeRuang,
    gedung,
    lantai,
    kapasitas,
    adaProyektor
  } = req.body;

  if (!kodeRuang || !gedung || !kapasitas === undefined) {
    return res.status(400).json({
      status: "error",
      message: "kodeRuang, gedung, dan kapasitas wajib diisi",
      data: null
    });
  }

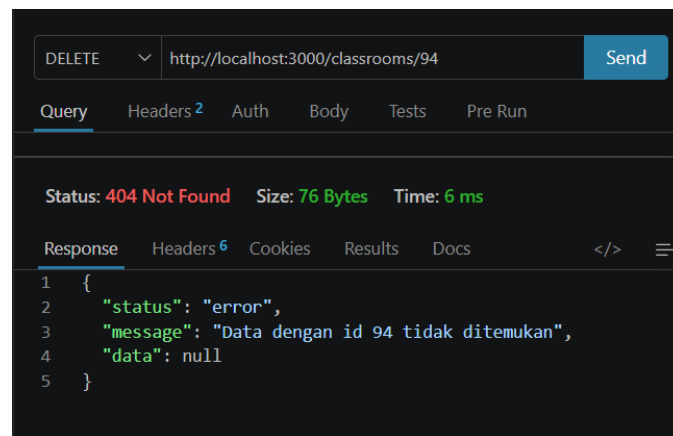
  const classroomDiubah = {
    id: id,
    kodeRuang: kodeRuang,
    gedung: gedung,
    lantai: lantai,
    kapasitas: kapasitas,
    adaProyektor: adaProyektor
  };
  classrooms[index] = classroomDiubah;
  res.status(200).json({
    status: "success",
    message: "Data ruang kelas berhasil diubah",
    data: classroomDiubah
  });
});
```

Gambar 14. Pengujian POST /classrooms dengan data tidak lengkap

Jika ID ditemukan, data dihapus dari array menggunakan splice(). Server kemudian memberikan status 200 dengan data bernilai null, Jika ID tidak ditemukan, server memberikan status 404.



Gambar 15. Pengujian DELETE /classrooms/1



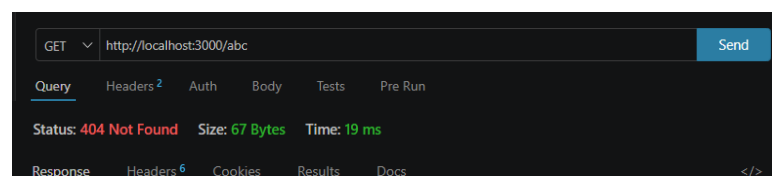
Gambar 16. Pengujian DELETE /classrooms/94

12. Membuat Middleware Endpoint 404

Setelah seluruh route dibuat, ditambahkan middleware untuk menangani request menuju endpoint yang tidak tersedia. Kode yang digunakan:

```
app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null  });
});
```

Middleware ini diletakkan setelah seluruh endpoint agar request yang tidak cocok dengan route sebelumnya dapat ditangani sebagai endpoint yang tidak ditemukan.



Gambar 17. Pengujian GET /abc sebagai endpoint yang tidak ditemukan

13. Menjalankan Server secara local

Server dijalankan menggunakan Node.js pada port 3000. Project menyediakan script start dan dev pada package.json.

Server dapat dijalankan menggunakan: `npm start` atau `npm run dev`

Jika berhasil, server dapat diakses melalui: <http://localhost:3000>

```
PS C:\Users\User\tugas1-restful-2428240072> npm start

> tugas1-restful-2428240072@1.0.0 start
> node app.js

Server berjalan di http://localhost:3000
PS C:\Users\User\tugas1-restful-2428240072> npm run dev

> tugas1-restful-2428240072@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Server berjalan di http://localhost:3000
```

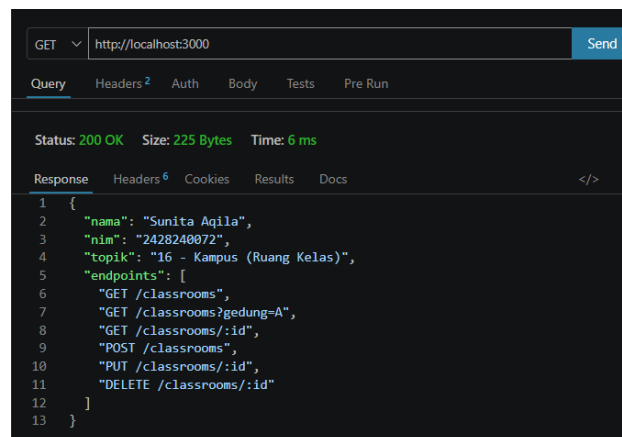
Gambar 18. Server Express.js berhasil berjalan pada terminal

14. Menguji API Menggunakan Thunder Client

Setelah server berjalan, dilakukan pengujian terhadap RESTful API menggunakan Thunder Client pada Visual Studio Code. Pengujian dilakukan dengan memasukkan method, URL, serta request body sesuai kebutuhan masing-masing endpoint.

Pengujian mencakup method GET, POST, PUT, dan DELETE, termasuk pengujian data berhasil, data tidak ditemukan, validasi field wajib, filter berdasarkan gedung, serta endpoint yang tidak tersedia.

Hasil dari pengujian dicatat berdasarkan HTTP status code dan response JSON yang diberikan oleh server. Rekapitulasi hasil pengujian akhir disajikan pada Bagian V.



```
GET http://localhost:3000

Status: 200 OK Size: 225 Bytes Time: 6 ms

Response
1 {
2   "nama": "Sunita Aqila",
3   "nim": "2428240072",
4   "topik": "16 - Kampus (Ruang Kelas)",
5   "endpoints": [
6     "GET /classrooms",
7     "GET /classrooms?gedung=A",
8     "GET /classrooms/:id",
9     "POST /classrooms",
10    "PUT /classrooms/:id",
11    "DELETE /classrooms/:id"
12  ]
13 }
```

Gambar 19. Contoh pengujian endpoint menggunakan Thunder Client

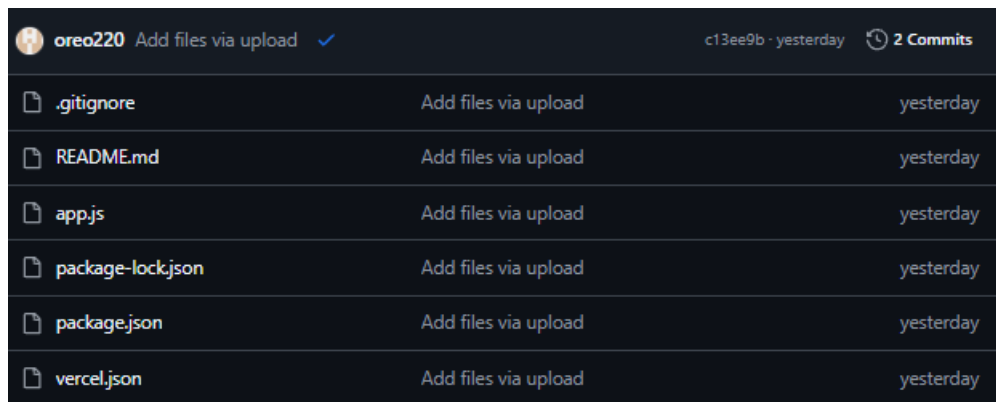
15. Membuat Repository GitHub

Setelah API selesai dan berhasil diuji secara lokal, project diunggah ke GitHub. Repository dibuat menggunakan format nama yang ditentukan dalam praktikum: `tugas1-restful-2428240072`

Repository dibuat dalam kondisi Public agar dapat diakses untuk keperluan penilaian. File yang diunggah meliputi:

1. app.js
2. package.json
3. package-lock.json
4. vercel.json
5. .gitignore
6. README.md

Folder node_modules tidak diunggah karena sudah dicantumkan pada .gitignore.



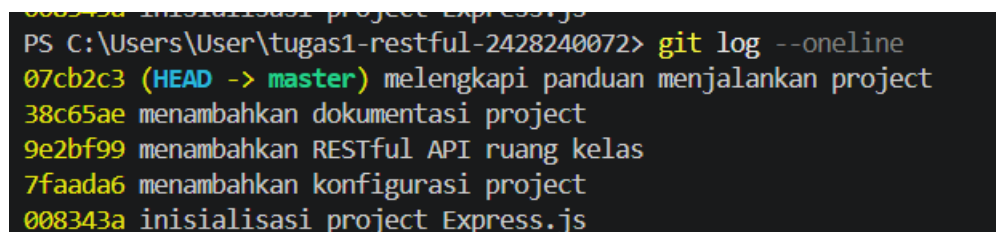
Gambar 20. Repository GitHub project

16. Membuat Commit Secara Bertahap

Setelah project selesai dikembangkan, dilakukan commit secara bertahap menggunakan Git. Commit dibuat untuk mencatat perkembangan project mulai dari inisialisasi, konfigurasi, pembuatan RESTful API, dokumentasi, hingga penyempurnaan panduan project. Riwayat commit dapat diperiksa menggunakan perintah:

`git log --oneline`

Riwayat tersebut digunakan sebagai bukti bahwa project dikerjakan secara bertahap.



Gambar 21. Hasil git log --oneline

17. Deployment ke Vercel

Setelah repository GitHub selesai, project di-deploy ke Vercel agar RESTful API dapat diakses melalui internet.

Project menggunakan file vercel.json dengan konfigurasi:

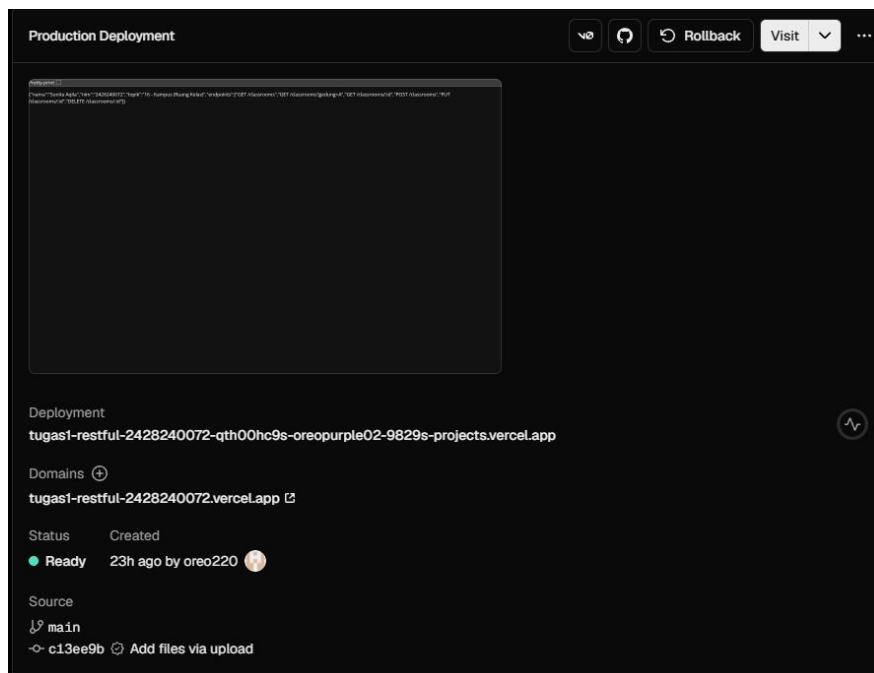
```
{
```

```

"version": 2,
"builds": [
  {
    "src": "app.js",
    "use": "@vercel/node"
  }
],
"routes": [
  {
    "src": "/(.*)",
    "dest": "app.js"
  }
]
}

```

Pada app.js, app.listen() hanya digunakan ketika aplikasi dijalankan secara lokal, sedangkan app diekspor untuk kebutuhan deployment.

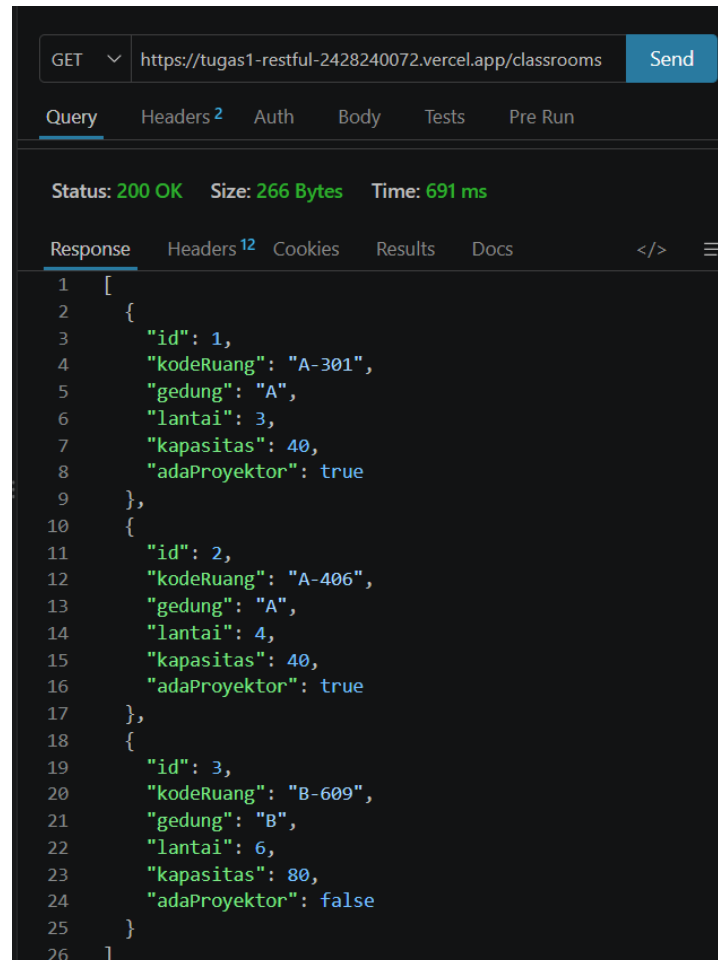


Gambar 21. Hasil Deploy ke Vercel

18. Pengujian API pada Vercel

Setelah deployment berhasil, dilakukan pengujian kembali menggunakan URL Vercel. Tahap ini bertujuan memastikan RESTful API dapat berjalan pada lingkungan deployment dan seluruh endpoint dapat diakses melalui internet. Pengujian akhir pada URL Vercel meliputi endpoint GET, POST, PUT, DELETE, filter berdasarkan gedung, data yang tidak ditemukan, validasi data, serta endpoint yang tidak tersedia.

Hasil pengujian akhir pada Vercel digunakan sebagai data pada tabel V. Hasil Pengujian.



Gambar 22. Pengujian API menggunakan URL Vercel pada Thunder Client

V. Hasil dan Pembahasan

Pengujian dilakukan untuk memastikan seluruh endpoint RESTful API ruang kelas dapat berjalan sesuai dengan rancangan. Pengujian meliputi proses membaca data, menambahkan data, mengubah data, menghapus data, melakukan filter berdasarkan gedung, serta menguji kondisi ketika data atau endpoint tidak ditemukan. Pengujian dilakukan menggunakan Thunder Client dengan mengirimkan request dan memeriksa response, status code, serta format JSON yang dihasilkan oleh server.

Hasil pengujian dirangkum pada tabel berikut.

No	Method	Endpoint	Data Request	Status Diharapkan	Status Hasil	Keterangan
1	GET	/classrooms	—	200	200	Sesuai

2	GET	/classrooms/1	—	200	200	Sesuai
3	GET	/classrooms/99	—	404	404	Sesuai
4	GET	/classrooms?gedung=A	—	200	200	Sesuai
5	POST	/classrooms	Body lengkap	201	201	Sesuai
6	POST	/classrooms	Field wajib kosong	400	400	Sesuai
7	PUT	/classrooms/1	Body lengkap	200	200	Sesuai
8	PUT	/classrooms/99	Body lengkap	404	404	Sesuai
9	DELETE	/classrooms/1	—	200	200	Sesuai
10	DELETE	/classrooms/99	—	404	404	Sesuai

Berdasarkan hasil pengujian, seluruh endpoint yang diuji menghasilkan status code sesuai dengan yang diharapkan. Response yang diberikan server juga menggunakan format JSON sesuai dengan ketentuan RESTful API pada praktikum. Method GET digunakan untuk mengambil data, POST untuk menambahkan data, PUT untuk mengubah data, dan DELETE untuk menghapus data. Validasi data menghasilkan status 400 apabila field wajib tidak lengkap, sedangkan data atau endpoint yang tidak ditemukan menghasilkan status 404.

VI. Kendala dan Solusi

Optional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1.	Pada saat akan melihat riwayat commit menggunakan perintah git log --oneline, muncul pesan bahwa folder project belum merupakan repository Git. Hal ini terjadi karena repository Git belum dibuat pada folder project.	Dilakukan inisialisasi repository menggunakan perintah git init, kemudian project dibuatkan commit secara bertahap hingga memenuhi ketentuan minimal lima incremental commit.
2.	Saat melakukan commit pertama, Git menampilkan pesan Author identity unknown karena nama dan email pengguna belum dikonfigurasi.	Dilakukan konfigurasi identitas Git menggunakan git config --global user.name dan git config --global user.email, kemudian proses commit dapat dilakukan kembali.
3.	Setiap endpoint memiliki fungsi dan status code yang berbeda sehingga perlu dipastikan	setiap endpoint diuji menggunakan Thunder Client, meliputi request berhasil, data atau ID yang tidak

	bahwa response yang diberikan sesuai dengan ketentuan tugas.	ditemukan, validasi field wajib, serta filter berdasarkan gedung. Hasil pengujian kemudian dibandingkan dengan status code yang diharapkan.
--	--	---

VII. Kesimpulan

Simpulkan hasil praktikum dikaitkan dengan tujuan pada bagian I.

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa RESTful API dapat dibangun menggunakan Express.js dengan memanfaatkan routing dan HTTP method sesuai fungsi masing-masing. Pada praktikum ini berhasil dibuat RESTful API untuk resource ruang kelas dengan endpoint /classrooms yang mendukung operasi GET, POST, PUT, DELETE, serta filter berdasarkan gedung.

API telah menggunakan format JSON dan menerapkan status code HTTP sesuai kondisi request, yaitu 200 untuk request berhasil, 201 untuk data yang berhasil ditambahkan, 400 untuk data yang tidak memenuhi validasi, dan 404 untuk data atau endpoint yang tidak ditemukan. Pengujian juga dilakukan menggunakan Thunder Client untuk memastikan setiap endpoint dapat memberikan response sesuai ketentuan.

Selain itu, project telah dikelola menggunakan Git dengan minimal lima incremental commit dan disiapkan untuk proses deployment pada Vercel. Dengan demikian, praktikum ini memberikan pemahaman mengenai pembuatan, pengujian, pengelolaan versi, dan deployment RESTful API sederhana menggunakan Express.js.

Dengan demikian, tujuan praktikum untuk memahami dasar pembuatan dan pengujian RESTful API menggunakan Express.js telah diterapkan melalui pembuatan endpoint serta pengujian request dan response.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Express.js. (t.thn.). *Routing*. Diambil kembali dari Express.js Documentation:

<https://expressjs.com/en/guide/routing/>

Express.js. (t.thn.). *Using middleware*. Diambil kembali dari Express.js Documentation:

<https://expressjs.com/en/guide/using-middleware/>

Express.js. (t.thn.). *Routing*. Diambil kembali dari Express.js Documentation:

<https://expressjs.com/en/guide/routing/>

Mozilla. (t.thn.). *HTTP messages*. Diambil kembali dari MDN Web Docs:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>

Mozilla. (t.thn.). *HTTP request methods*. Diambil kembali dari MDN Web Docs:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

Mozilla. (t.thn.). *JSON*. Diambil kembali dari MDN Web Docs:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

Node.js. (t.thn.). *Introduction to Node.js*. Diambil kembali dari Node.js:

<https://nodejs.org/learn/getting-started/introduction-to-nodejs>

Smalley, P. P. (2025, April 24). *What Is a REST API (RESTful API)?* Diambil kembali dari

IBM: <https://www.ibm.com/think/topics/rest-apis>

Lampiran

Tautan Repository GitHub:

<https://github.com/oreo220/tugas1-restful-2428240072>

Tautan Deployment :

<https://github.com/oreo220/tugas1-restful-2428240072>

Riwayat Commit :

Riwayat commit digunakan untuk menunjukkan proses pengembangan project secara bertahap. Berdasarkan hasil git log --oneline, terdapat lima commit pada repository project, yaitu:

1. 008343a — inisialisasi project Express.js
2. 7faada6 — menambahkan konfigurasi project
3. 9e2bf99 — menambahkan RESTful API ruang kelas
4. 38c65ae — menambahkan dokumentasi project
5. 07cb2c3 — melengkapi panduan menjalankan project

```
PS C:\Users\User\tugas1-restful-2428240072> git log --oneline
07cb2c3 (HEAD -> master) melengkapi panduan menjalankan project
38c65ae menambahkan dokumentasi project
9e2bf99 menambahkan RESTful API ruang kelas
7faada6 menambahkan konfigurasi project
008343a inisialisasi project Express.js
```

Gambar 23. Riwayat commit menggunakan git log --oneline